Wardrobe IQ

System Architecture — Taste-Aware Outfit Utility Engine

Taste Inference · Outfit Utility · Augmented Discovery · Evaluation

① DATA INPUTS

VISUAL TASTE SIGNAL (COLD START)

📌 Pinterest Board URL
Paste link → scrape board images

🖼️ Image Upload
10 reference images → aesthetic fingerprint

Why this, not a quiz
Users already curate taste on Pinterest. Import captures aspiration — not just purchase history. Produces meaningful cold-start signal from 0 behavioral data.

BEHAVIORAL SIGNAL (WARM STATE)

💾 Phia Saves
Existing saves = working wardrobe. No manual cataloging.

📊 Interaction History
Clicks, dwell time, clickouts, price tier engagement

Key design decision
Wardrobe = Phia saves, not physical closet. Zero friction. Signal accumulates passively. Every save improves recommendation quality.

feeds into ▼

② TASTE EXTRACTION PIPELINE

AESTHETIC PROFILE BUILDER

🧠 CLIP Encoder
ViT-B/32. Encodes each input image into 512-dim aesthetic embedding.

⚖️ Embedding Pooling
Recency-weighted mean. Pinterest images weight 0.6, saves weight 1.0.

🗺️ UMAP + HDBSCAN
Reduces embedding space. Identifies dominant aesthetic clusters.

🏷️ Attribute Extraction
From cluster centroids: silhouette, color story, formality, occasion, price tier.

Silhouette	Color Story	Formality	Occasion	Price Tier
Fitted / Relaxed / Oversized / Structured	Neutral / Warm / Cool / Mixed / Monochrome	Casual — Smart Casual — Business — Formal	Daily / Work / Evening / Athletic / Travel	Inferred from save metadata + brand signals

Output: taste_vector $\in \mathbb{R}^{512}$ + structured aesthetic_attributes JSON. Updated incrementally on each new save.

produces taste_vector → feeds ranker ▼

③ OUTFIT UTILITY ENGINE — THE CORE ML SYSTEM

CANDIDATE GENERATION

🕳️ Outfit Slot Gap Analysis
Identify underrepresented slots in saves: Tops / Bottoms / Outerwear / Shoes / Bags / Accessories

🔍 Taste-Filtered ANN Retrieval
Query catalog using taste_vector. Filter to gap slot categories. Top-100 candidates per slot.

💰 Price Band Filter
Hard filter to ±30% of inferred price tier. Soft decay beyond range.

Separate candidate pool per slot. Never full-catalog search. Constrained retrieval = quality over recall.

CATEGORY-CONDITIONED COMPATIBILITY MODEL

Scoring Formula

```
score(c) =
  c_emb^T · W_AB · anchor_emb
  × taste_compat(c, taste_vec)
  × outfit_unlock_count(c, saves)
```

W_AB = learned compatibility matrix for category pair (A→B). Initialized from editorial rules, fine-tuned on co-save data.

⚠️ Why not cosine similarity?
CLIP cosine = visual similarity. Outfit compatibility ≠ similarity. A blazer + turtleneck are similar but redundant. A blazer + wide-leg trouser are different but compatible. W_AB captures directionality.

Outfit unlock count: for each candidate, count how many new complete outfit permutations (top+bottom+shoe, etc.) become possible across existing saves. Primary ranking signal.

RANKING & OUTPUT ASSEMBLY

🏆 Final Score Composition
w1·taste_compat + w2·outfit_unlock + w3·value_score. Weights configurable per surface type.

🔧 Post-Processing
Dedupe across slots. Category diversity enforcement. Confidence gating — suppress if top score below threshold.

💬 Explanation Generation
Template-based from top contributing features. Never freeform LLM. "Pairs with your saved trouser — unlocks 5 new looks."

Output: ranked list of (item, unlock_count, taste_score, explanation, confidence). One unified payload. Server-side orchestration only.

recommendations → surfaces ▼

④ PRODUCT SURFACES

AUGMENTED "SHOULD I BUY THIS?" PANEL

Extends Phia's existing core interaction. Same button. Richer response.

Demo Panel — Phia × Wardrobe IQ

94%	+7	$89
Taste Fit vs your aesthetic	Outfit Unlocks with your saves	Best Price resale via Phia

"Pairs with your saved linen trouser and 6 other saves — highest outfit utility in your wardrobe this week."

Phia's price comparison is unchanged. Taste fit + outfit unlocks are additive panels. No existing functionality removed.

WARDROBE TAB — UTILITY VIEW

Persistent home for saves. Surfaces outfit utility across the whole wardrobe.

🔥 Outfit Unlock Heatmap
Saved items ranked by how many looks they enable. High-utility items highlighted. Low-utility items surfaced with "consider pairing with X."

🎯 Gap Recommendation
"Your wardrobe is strong in tops and outerwear. Adding one versatile bottom unlocks 12 new combinations." Single, specific recommendation only.

✨ Complete the Look
Select any saved item → see which specific other saves it pairs with + the one recommended addition that completes the set.

logs interactions → ▼

⑤ COLD START STRATEGY & ⑥ EVALUATION FRAMEWORK

⑤ COLD START STRATEGY

- **0 saves** — Visual taste extraction only. Recommendations use taste_vector as sole signal. Item aesthetic cluster as ranking prior.
- **1–4 saves** — Hybrid: taste_vector (70%) + save embeddings (30%). Gap analysis begins. Basic outfit utility scoring active.
- **5–14 saves** — Save embeddings dominate (60%). Outfit unlock count is meaningful. Category coverage analysis reliable.

⑥ EVALUATION FRAMEWORK

Three evaluations with clear hypotheses. This separates a project from a demo.

❄️ Cold Start Preference Prediction
Hypothesis: visual taste extraction predicts held-out saves better than popularity baseline at 0, 3, 5 saves. Metric: Precision@10 vs baseline.

15+ saves Full system. Strong wardrobe embedding. Utility scoring confident. Trend signal boost active as tertiary feature.

📐 Outfit Utility Calibration

Hypothesis: high outfit-unlock-count items are co-saved more often in same-wardrobe sessions. Metric: Spearman correlation between predicted utility and observed co-save rate.

🎯 Taste Fit AUC-ROC

Hypothesis: taste_compat score predicts user saves vs skips in held-out evaluation set. Metric: AUC-ROC across simulated user profiles with varied aesthetic clusters.

⑦ TECH STACK & ⑧ EXPLICIT SCOPE CUTS

⑦ TECH STACK — INDEPENDENTLY BUILDABLE

CLIP ViT-B/32
Fashion aesthetic embeddings. Open weights. Zero API cost.

UMAP + HDBSCAN
Aesthetic cluster extraction from image corpus. CPU-only.

FAISS (local)
ANN vector search. Free, no infra needed at demo scale.

FastAPI
Recommendation orchestrator. Lightweight, async.

Next.js + Tailwind
Demo frontend styled as Phia panel.

SerpApi + Supabase
Public product catalog (~$5 one-time) + free tier logging DB.

Total estimated build cost: ~$5–10. All ML inference is local CPU. No hot-path LLM calls.

⑧ EXPLICIT SCOPE CUTS

Discipline on scope is part of the signal you're sending.

- ✕ **Physical closet import** — Users won't do it. Friction kills the product.
- ✕ **Redundancy flagging** — Anti-revenue. Outfit utility scoring handles it implicitly.
- ✕ **Creator-to-cart layer** — Good idea. Future extension. Not MVP.
- ✕ **Social / follow features** — Phia's roadmap item, not yours to build.
- ✕ **Browser extension** — Engineering distraction. Web demo is sufficient.
- ✕ **Trend forecasting pipeline** — Too many dependencies. Trend is one feature, not the product.
- ✕ **Freeform LLM explanations** — Prompt wrapper risk. Template-based only in v0.

WHY THIS LANDS WITH PHIA — BUSINESS ALIGNMENT

Series A Roadmap Match

Their published roadmap explicitly names "taste-aware recommendations" and "community-driven digital closets." This builds both simultaneously.

Return Rate Story

Their commercial claim is 50%+ return rate reduction. Outfit utility scoring — buying pieces that work with existing saves — directly reduces purchase regret, the primary return driver.

Personalization Cold Start

Their proprietary LLM requires transaction history. Visual taste extraction works from zero. Directly solves the hardest open problem in their personalization roadmap.